

Operating Systems



UNIT 2

Process Management

Process Concept, States & Control, Threads, Uni-processor Scheduling, System Calls



Process Concept

A process is a program in execution — an active entity, whereas a program is a passive entity (just code on disk)

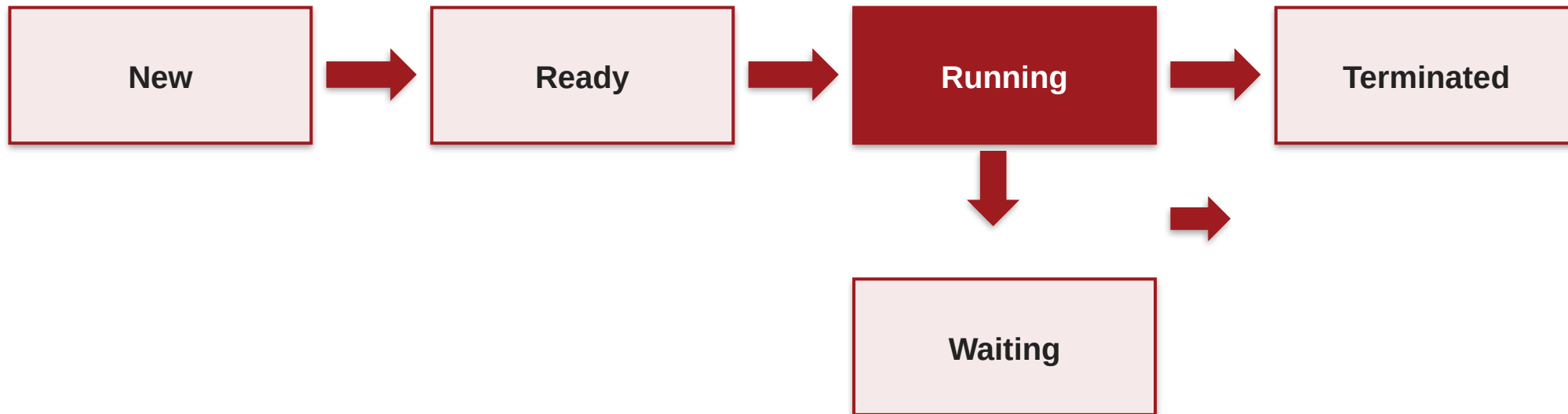
A process includes: the program code (text section), current activity (program counter, registers), stack, data section, and heap

Multiple processes may be associated with the same program, executing different parts of the code or run by different users

The OS is responsible for creating, scheduling and terminating processes



Process States



New -> Ready: admitted. Ready -> Running: dispatched by scheduler. Running -> Waiting: I/O or event wait. Waiting -> Ready: event completion. Running -> Ready: preempted (time quantum expired). Running -> Terminated: process exits.

Process Control Block (PCB)

The PCB is a data structure maintained by the OS for every process; it stores all information needed to manage and resume that process

Process State — new, ready, running, waiting, terminated

Process Number (PID) — unique process identifier

Program Counter — address of the next instruction to execute

CPU Registers — accumulators, index registers, stack pointers, etc.

CPU Scheduling Information — priority, scheduling queue pointers

Memory Management Information — base/limit registers, page tables

Accounting & I/O Status Information — CPU used, time limits, open files, allocated devices



Process Control

Process Creation — a parent process creates child processes, forming a tree of processes; resources may or may not be shared

Process Termination — a process executes its last statement and requests deletion via an `exit()` system call

Context Switching — saving the state of the current process (in its PCB) and loading the saved state of the next process

Process Hierarchy — parent-child relationships identified by Process ID (PID) and Parent Process ID (PPID)

Cascading Termination — in some systems, if a parent terminates, all its children are terminated as well



Threads: Concept

A thread is a basic unit of CPU utilization — a lightweight process

A traditional (heavyweight) process has a single thread of control; modern operating systems support multithreaded processes

Threads within the same process share the code section, data section, and OS resources (open files, signals)

Each thread has its own thread ID, program counter, register set, and stack

Benefits: responsiveness, resource sharing, economy, and scalability across multiprocessor architectures



Types of Threads & Multithreading Models

User-Level Threads — managed by a user-level thread library without kernel support; fast to create and manage, but the whole process blocks if one thread blocks

Kernel-Level Threads — managed directly by the OS kernel; the kernel can schedule threads individually across multiple CPUs

Multithreading models map user threads to kernel threads:

Many-to-One — many user threads mapped to a single kernel thread

One-to-One — each user thread mapped to its own kernel thread

Many-to-Many — many user threads multiplexed onto a smaller or equal number of kernel threads



Uni-processor Scheduling: Overview

CPU scheduling decides which process in the ready queue is allocated the CPU next

Long-Term Scheduler (Job Scheduler) — selects processes from disk and loads them into memory, controlling the degree of multiprogramming

Short-Term Scheduler (CPU Scheduler) — selects a process from the ready queue and allocates the CPU; runs very frequently

Medium-Term Scheduler — swaps processes in and out of memory to manage the multiprogramming level

Dispatcher — gives control of the CPU to the process chosen by the short-term scheduler (context switch, mode switch)



Types of Scheduling: Preemptive vs Non-Preemptive

Preemptive Scheduling

The CPU can be taken away from a running process before it completes, e.g. when a higher-priority process arrives

Necessary for time-sharing and interactive systems to ensure fair response time

Requires protecting shared data since a process can be interrupted mid-update (synchronization needed)

Examples: Round Robin, SRTF, Priority (preemptive)

Non-Preemptive Scheduling

Once the CPU is allocated to a process, it keeps the CPU until it terminates or switches to the waiting state

Simpler to implement; no forced context switches

Can lead to poor response time if a long process is currently running

Examples: FCFS, SJF (non-preemptive), Priority (non-preemptive)



Scheduling Criteria

CPU Utilization — keep the CPU as busy as possible

Throughput — number of processes completed per unit time

Turnaround Time — total time from submission to completion of a process

Waiting Time — total time a process spends waiting in the ready queue

Response Time — time from submission until the first response is produced (critical for interactive systems)



FCFS Scheduling (First-Come, First-Served)

Processes are executed strictly in the order they arrive in the ready queue

Implemented using a simple FIFO queue; non-preemptive in nature

Easy to understand and implement

Major disadvantage — Convoy Effect: short processes wait behind one long process, increasing average waiting time

Example: P1 (burst 24), P2 (burst 3), P3 (burst 3) arriving together -> order P1, P2, P3 gives P1 wait 0, P2 wait 24, P3 wait 27



SJF Scheduling (Shortest Job First)

Selects the process with the smallest next CPU burst time

Can be preemptive — Shortest Remaining Time First (SRTF) — or non-preemptive

Provably optimal: gives the minimum average waiting time for a given set of processes

Disadvantage: the exact length of the next CPU burst is rarely known in advance — usually estimated

Can cause starvation of longer processes if short processes keep arriving



Round Robin (RR) Scheduling

Each process gets a small, fixed unit of CPU time called a time quantum (typically 10-100 ms)

The ready queue is treated as a circular queue; the scheduler cycles through it, giving each process one quantum

Preemptive by design — if a process doesn't finish within its quantum, it is preempted and placed at the back of the queue

Performance depends heavily on the time quantum size — too large behaves like FCFS, too small increases context-switch overhead

Well suited for time-sharing and interactive systems since every process gets regular CPU turns



Priority Scheduling

Each process is assigned a priority; the CPU is allocated to the process with the highest priority

Can be implemented as either preemptive or non-preemptive

SJF is a special case of priority scheduling where priority is the inverse of the predicted next CPU burst

Major problem — Starvation: low-priority processes may wait indefinitely

Solution — Aging: gradually increase the priority of processes that have waited a long time



Comparison of Scheduling Algorithms

Strengths

FCFS: simple, fair arrival order, no starvation

SJF: minimum average waiting time

Round Robin: fair, good response time, no starvation

Priority: important tasks handled first

Weaknesses

FCFS: convoy effect, poor for short jobs behind long ones

SJF: needs burst-time prediction, can starve long jobs

Round Robin: performance very sensitive to quantum size

Priority: can starve low-priority processes (needs aging)



Thread Scheduling

On systems supporting threads, it is kernel-level threads — not processes — that the OS actually schedules

Process-Contention Scope (PCS) — a thread library schedules user-level threads onto an available kernel thread

System-Contention Scope (SCS) — the kernel schedules kernel-level threads directly onto a CPU

Local Scheduling — the thread library decides which user thread runs on an available kernel thread

Global Scheduling — the kernel decides which kernel thread runs on the CPU next



Real-Time Scheduling

Real-time scheduling must guarantee that critical tasks meet their deadlines

Rate Monotonic Scheduling — a static-priority algorithm; tasks with shorter periods get higher priority

Earliest Deadline First (EDF) — a dynamic-priority algorithm; the task with the closest deadline gets the highest priority

Priority Inversion — a lower-priority task holds a resource needed by a higher-priority task, blocking it; solved using priority-inheritance protocols

Widely used in embedded systems, industrial control, and avionics



System Calls: ps, fork, exec

ps — a user command (built on system calls) that reports information on currently running processes by reading kernel process tables

fork() — creates a new process (child) by duplicating the calling parent process; returns 0 to the child and the child's PID to the parent

exec family (execl, execv, execve, execvp, ...) — replaces the current process image with a new program; does not create a new process

fork() followed by exec() is the standard Unix pattern for running a new program in a child process



System Calls: wait, join

`wait()` — makes the parent process pause until one of its child processes terminates; also collects the child's exit status

`waitpid()` — a more flexible version of `wait()` that can wait for a specific child process

`pthread_join()` — blocks the calling thread until the specified target thread terminates; used to collect a thread's return value

Failing to call `wait()/join()` can leave zombie processes or unclaimed thread resources behind



Unit 2 Summary

A process is a program in execution, described and tracked by its Process Control Block (PCB)

Processes move through states: New, Ready, Running, Waiting, and Terminated

Threads are lightweight units of execution within a process, sharing its resources

Scheduling can be preemptive or non-preemptive; judged by utilization, throughput, turnaround, waiting and response time

Key algorithms: FCFS, SJF, Round Robin, Priority; real-time scheduling adds strict deadline guarantees

System calls like fork, exec, wait and pthread_join manage the process/thread lifecycle

